

RSA vs ECDHE – Kulcscsere és Shared Secret működése

Oktató tananyag mérnök hallgatók számára

1. Bevezetés – Miért van szükség kulcscserére?

A szimmetrikus titkosítás gyors és hatékony, de szükség van egy közös titkos kulcsra. Nyílt hálózaton ezt nem küldhetjük el egyszerűen, mert lehallgatható. A kulcscsere célja: közös titok létrehozása anélkül, hogy azt elküldenénk.

Ábra: TLS kontextus

```
Client                                Server
|----- ClientHello ----->|
|<----- ServerHello -----|
|           Kulcscsere           |
|===== Titkos csatorna =====>
```

2. RSA alapú kulcscsere

RSA esetén a kliens generál egy Pre-Master Secret értéket, amit a szerver nyilvános kulcsával titkosít, majd elküld.

RSA lépések

1. Client generál: PreMasterSecret
2. Client titkosítja RSA public key-jel
3. Server visszafejti private key-jel
4. Mindkét fél kiszámolja a MasterSecret-et

Ábra: RSA kulcscsere folyamata

```
Client                                Server
|--- PreMasterSecret (RSA encrypt) ---->|
|                                         |
|<----- (Decrypt private key) -----|
|                                         |
Shared secret = PRF(PreMasterSecret)
```

A szerver a saját privát kulcsával visszafejti a titkot. Ezután mindkét fél ugyanabból a Pre-Master Secretből generálja a session kulcsokat.

Hátrány: ha a szerver privát kulcsa később kompromittálódik, a korábban rögzített forgalom visszafejthető (nincs Perfect Forward Secrecy).

3. Diffie–Hellman alapgondolat

A Diffie–Hellman módszerben mindkét fél saját titkos számot választ, majd egy közös matematikai művelet révén ugyanarra az eredményre jut.

Client: $A = g^a \bmod p$

Server: $B = g^b \bmod p$

Shared secret:

Client: $S = B^a = g^{(ab)}$

Server: $S = A^b = g^{(ab)}$

4. ECDHE működése

Az ECDHE a Diffie–Hellman elvet alkalmazza elliptikus görbéken. Kisebb kulcsméret mellett azonos biztonságot ad.

ECDHE lépések

1. Client választ: a
2. Client számolja: $A = a * G$
3. Server választ: b
4. Server számolja: $B = b * G$
5. Publikus kulcs csere (A, B)
6. Shared secret:
Client: $S = a * B$
Server: $S = b * A$

Mindkét fél generál egy privát számot (a és b). A generátor pont (G) egy előre definiált pont az elliptikus görbén, amelyből minden publikus kulcs számítható. Mérnöki értelemben: G egy közös referencia pont, amelyből a privát szám segítségével egyedi publikus pont állítható elő.

Ábra: ECDHE azonosság

Client számolja: $S = a * (b * G)$

Server számolja: $S = b * (a * G)$

Mindkettő eredménye: $(a * b) * G$

A privát értékek soha nem kerülnek átvitelre. Minden kapcsolat új kulcsot használ (Ephemeral), ezért van Perfect Forward Secrecy.

5. Shared Secret → Session Key (HKDF)

A nyers shared secretből kulcsgeneráló függvény (HKDF) segítségével állítjuk elő a konkrét titkosítási kulcsokat.

Extract:

```
PRK = HMAC(salt, shared_secret)
```

Expand:

```
OKM = HMAC(PRK, info || counter)
```

6. Teljesítmény összehasonlítás

RSA 2048 bites moduláris exponenciálást használ. ECDHE 256 bites elliptikus görbe műveleteket használ. Az elliptikus görbe kriptográfia kisebb számok (kisebb kulcsméret) mellett ugyanolyan biztonságot nyújt, ezért gyorsabb és hatékonyabb.

7. Összehasonlító táblázat

Tulajdonság	RSA	ECDHE
Kulcsméret	2048 bit	256 bit
Forward Secrecy	Nem	Igen
CPU terhelés	Magas	Alacsonyabb
TLS 1.3 támogatás	Nem	Kötelező

8. Összefoglalás

RSA esetén a kliens generál és elküldi a titkot titkosítva. ECDHE esetén a felek matematikailag állítanak elő közös titkot anélkül, hogy azt elküldenék. Az ECDHE gyorsabb, biztonságosabb és modern TLS rendszerekben kötelező.